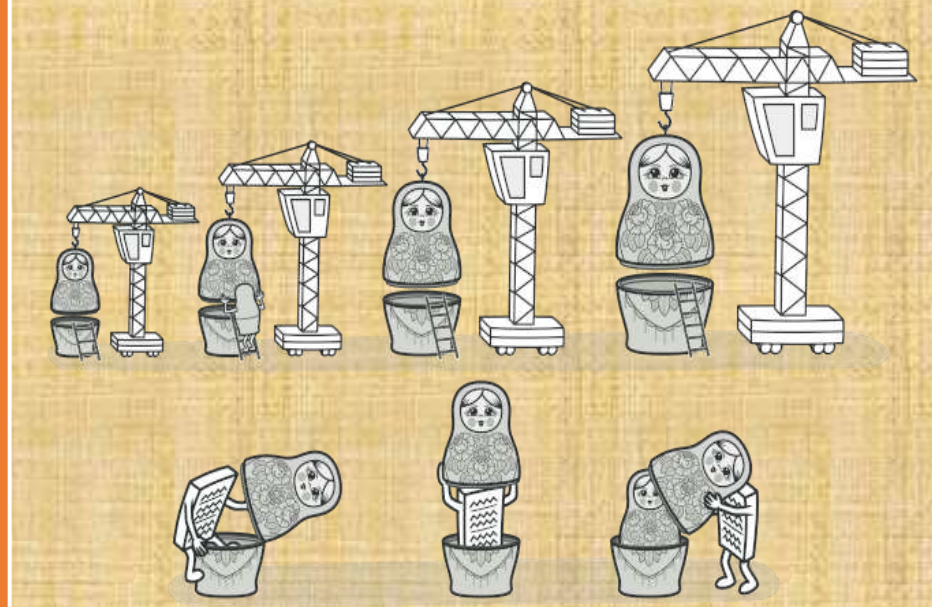
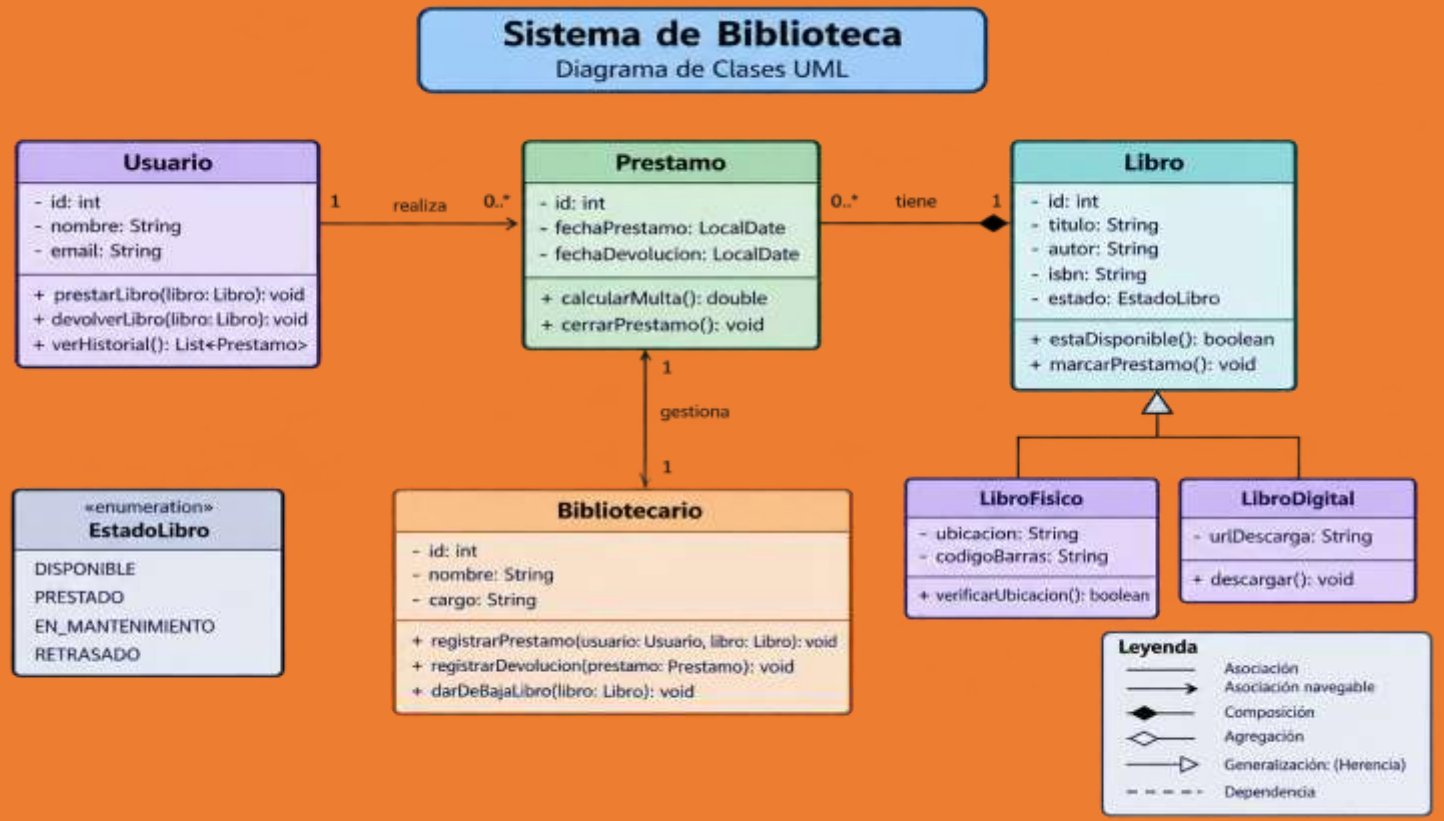
Patrón de Diseño: Decorador

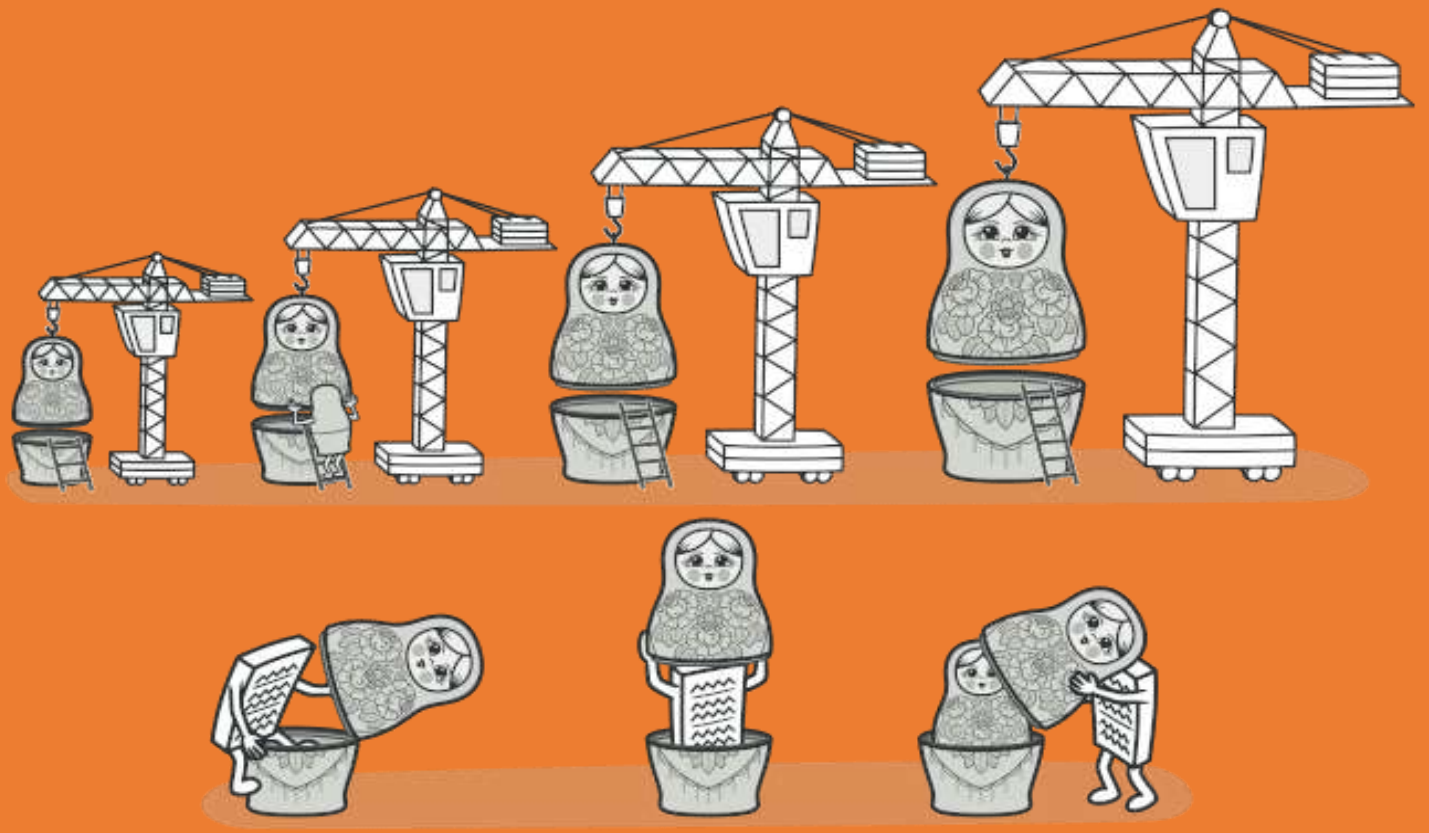
- Alumno: Benjamin Ezequiel Duarte Madana
- Profesor: Santos Silva Martin Fidel
- Materia: Alagoritmos y Estructura de Datos
- Instituto: Instituto Superior de Formacion Docente y Tecnica Nº 59



Introducción: ¿Qué es un patrón de diseño?



Patrón de Diseño: Decorador



¿Qué problema resuelve ?

- entidadConcreta(original)
- entidadConCasco()
- entidadConPechera()
- entidadConGuante()





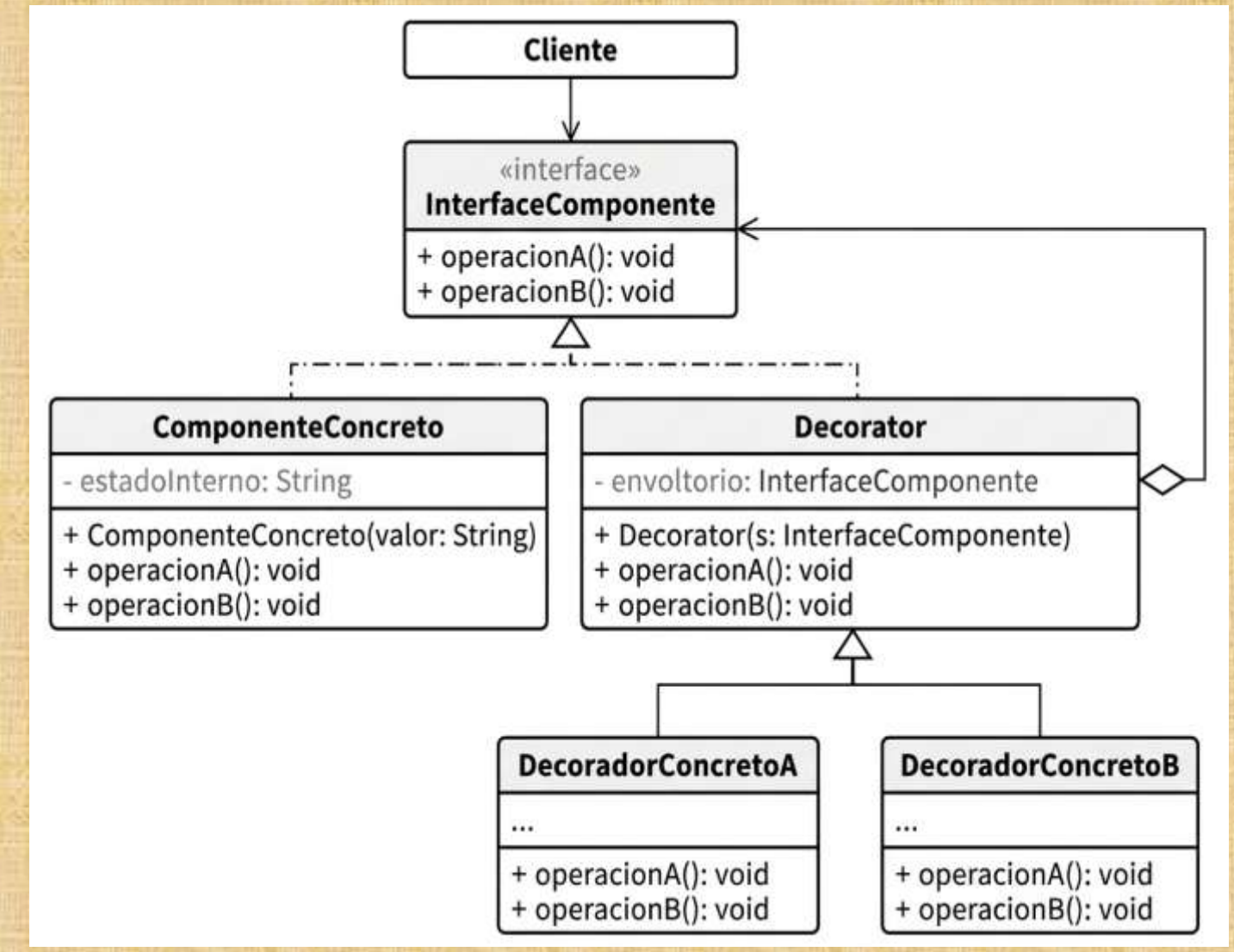
Problemas puntuales que soluciona

PROBLEMA		SOLUCIÓN CON DECORATOR	
Explosión de Subclases (Rigidez de la herencia)		Evita crear una subclase para cada combinación. Permite combinar características dinámicamente.	
Inflexibilidad en Tiempo de Ejecución		Añade o quita responsabilidades mientras la aplicación se ejecuta.	
Violación del Principio de Responsabilidad Única		Separa las responsabilidades principales de las secundarias o de infraestructura.	
Modificación de Código Existente		Extiende el comportamiento de clases sin alterar su código fuente original.	
Alternativa a la Herencia Múltiple		Permite componer un objeto con múltiples capacidades sin los problemas de la herencia múltiple.	

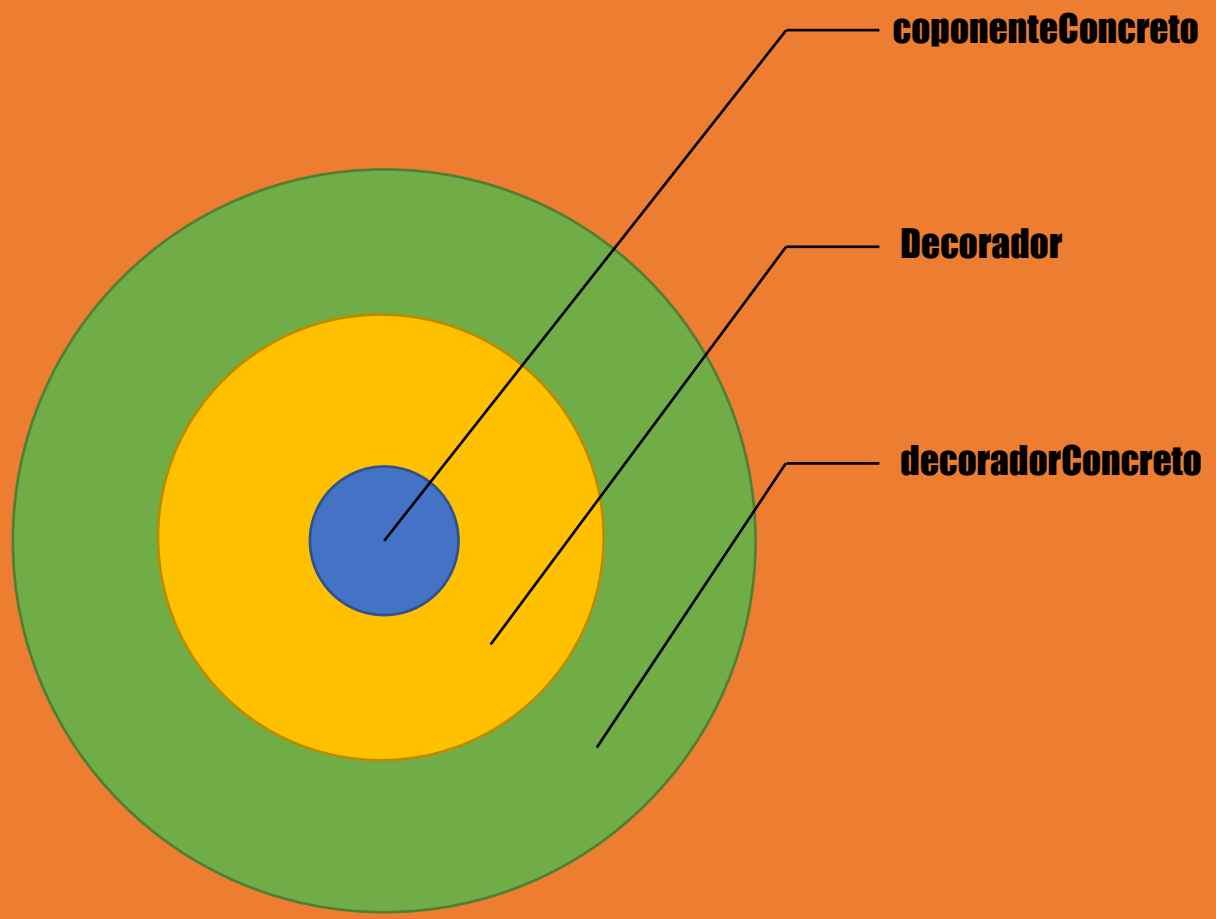
Dolores que sufre el patrón de diseño

DOLOR DE CABEZA		CONSECUENCIA EN PRODUCCIÓN
Debugging Infernal		Múltiples capas ocultan el objeto base dificultan rastrear errores
Pérdida de Identidad		El cliente interactúa con el wrapper rompiendo comparaciones directas
Inicialización Compleja		Requiere envoltura manual recursiva repetitiva en código
Explosión de Clases		Cada regla de negocio nueva exige crear un archivo separado
Interfaces Frágiles		Si la interfaz raíz cambia obliga a refactorizar todos los decoradores

Diagrama genérico uml del patrón de diseño decorator



También se le puede conocer como: Wrapper



Cómo implementarlo en java

Paso 1: Crear la interfaz Raiz (interfaceComponente)

- Este es el contrato que define lo que todos los objetos (tanto los básicos como los decoradores) pueden hacer:



Paso 2: crear el objeto real (componenteConcreto)

Esta es la clase base que realiza el trabajo real. No sabe nada de decoradores . Como indica tu diagrama, inclue un: estadoInterno

```java
// Archivo: ComponenteConcreto.java
public class ComponenteConcreto implements InterfaceComponente {
    private String estadoInterno;

    // Constructor que recibe el valor inicial
    public ComponenteConcreto(String valor) {
        this.estadoInterno = valor;
    }

    @Override
    public void operacionA() {
        System.out.println("ComponenteConcreto: Ejecutando operacionA. Estado = " + estadoInterno);
    }

    @Override
    public void operacionB() {
        System.out.println(x: "ComponenteConcreto: Ejecutando operacionB.");
    }
}
```

Paso 3: Crear la base de los decoradores (Decorator)

Esta es una clase abstracta muy importante. Cumple una doble función: implementa la interfaz y contiene (agrega) una referencia a un objeto de esa misma interfaz (el atributo envoltorio). Por defecto, solo redirige las llamadas al objeto que tiene guardado

```
1
2   // Archivo: Decorator.java
3   public abstract class Decorator implements InterfaceComponente {
4       // El rombo del diagrama: la referencia al objeto envuelto
5       protected InterfaceComponente envoltorio;
6
7       // Constructor que recibe el componente a decorar
8       public Decorator(InterfaceComponente s) {
9           this.envoltorio = s;
10      }
11
12      // Delegan el comportamiento por defecto al envoltorio
13      @Override
14      public void operacionA() {
15          envoltorio.operacionA();
16      }
17
18      @Override
19      public void operacionB() {
20          envoltorio.operacionB();
21      }
22  }
23
24
25
```

Pasa 4: Crear los decoradores específicos(decoradores concretos)

Estas clases heredan de Decorator modifican el comportamiento de los métodos. Pueden añadir su propia lógica antes o después de llamar al comportamiento original .

```java
// Archivo: DecoradorConcretoA.java
public class DecoradorConcretoA extends Decorator {

    public DecoradorConcretoA(InterfaceComponente s) {
        super(s); // Pasa el objeto al constructor de Decorator
    }

    @Override
    public void operacionA() {
        super.operacionA(); // Llama al comportamiento base/anterior
        comportamientoExtraA(); // Añade la nueva funcionalidad
    }

    private void comportamientoExtraA() {
        System.out.println(x: "-> [Decorador A]: Funcionalidad extra añadida a la operación A.");
    }
}

```

```java
// Archivo: DecoradorConcretoB.java
public class DecoradorConcretoB extends Decorator {

    public DecoradorConcretoB(InterfaceComponente s) {
        super(s);
    }

    @Override
    public void operacionB() {
        // En este ejemplo, el comportamiento extra se ejecuta ANTES del original
        comportamientoExtraB();
        super.operacionB();
    }

    private void comportamientoExtraB() {
        System.out.println(x: "-> [Decorador B]: Alerta! Preparando entorno antes de operación B.");
    }
}

```

Paso 5: El código del cliente (Puesta en marcha)

Aquí vemos cómo el cliente combina y apila los objetos de forma dinámica en tiempo de ejecución.

```java
// Archivo: Cliente.java
public class Cliente {
    Run | Debug
    public static void main(String[] args) {

        System.out.println(x: "--- 1. OBJETO BASE SIMPLE ---");
        InterfaceComponente componenteSimple = new ComponenteConcreto(valor: "ConfiguraciónInicial");
        componenteSimple.operacionA();
        componenteSimple.operacionB();

        System.out.println(x: "\n--- 2. OBJETO CON DECORADOR A ---");
        // Envolvemos el objeto simple dentro del Decorador A
        InterfaceComponente conDecoradorA = new DecoradorConcretoA(componenteSimple);
        conDecoradorA.operacionA(); // Ejecutará la base + el extra de A

        System.out.println(x: "\n--- 3. OBJETO TOTALMENTE DECORADO (A + B) ---");
        // Envolvemos el objeto ya decorado dentro del Decorador B
        InterfaceComponente completamenteDecorado = new DecoradorConcretoB(conDecoradorA);

        // Ejecuta todo el flujo acumulado
        completamenteDecorado.operacionA();
        completamenteDecorado.operacionB();
    }
}
```

¿Qué verás en la consola al ejecutarlo?

--- 1. OBJETO BASE SIMPLE ---
ComponenteConcreto: Ejecutando operacionA. Estado = ConfiguraciónInicialComponenteConcreto: Ejecutando operacionB.
--- 2. OBJETO CON DECORADOR A ---
ComponenteConcreto: Ejecutando operacionA. Estado = ConfiguraciónInicial-> [Decorador A]: Funcionalidad extra añadida a la operación A.
--- 3. OBJETO TOTALMENTE DECORADO (A + B) ---
ComponenteConcreto: Ejecutando operacionA. Estado = ConfiguraciónInicial-> [Decorador A]: Funcionalidad extra añadida a la operación A.-> [Decorador B]: Alerta! Preparando entorno antes de operación B.ComponenteConcreto: Ejecutando operacionB.